

Identify and debug

1. Course Content

Note: This program runs on a sandbox map. For personal maps, you'll need to adjust the program yourself. This course is for debugging the confidence level of the car in recognizing each road sign; the default settings do not require modification.

Basic: Start the autonomous driving program and observe the road sign recognition results.

2. Pre-Test Preparation

- Adjust Test Parameters

Parameter Path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/yolo_param.yaml
```

```
#Road sign detection inference parameters
sign_detection:
  model_path:
    '/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/sign_model.e
ngine'
  conf: 0.92          #Sets the minimum confidence threshold for detection.
                      Detected objects with a confidence level below this threshold will be ignored.
                      Adjusting this value helps reduce false positives.
  iou: 0.7           #An overlap threshold is used for non-maximum suppression
                      (NMS). Lower values ••reduce detections by eliminating overlapping bounding
                      boxes.
  rect: True         #If enabled, the shorter side of the image is minimally padded
                      until it is divisible by the step size to improve inference speed.
  half: True         #If enabled, half-precision inference is used, which can speed
                      up model inference on supported GPUs with minimal impact on accuracy.
  device: "cuda:0"   #Reasoning device
  batch : 50         #Batch size: Larger batch sizes can provide higher
                      throughput, thereby reducing the total time required for inference.
  max_det : 2        #Maximum number of objects detected
  augment : False    #Enabling Test-Time Enhancement (TTA) for prediction
                      may improve the robustness of detection, but it will reduce inference speed.
  verbose : False    #Whether to output detailed inference information to
                      the terminal.
  classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]

line_detection:
#Lane detection inference parameters
  model_path:
    '/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/tools/lane_v6.engi
ne'
  conf: 0.5          #Sets the minimum confidence threshold for detection. Detected
                      objects with a confidence level below this threshold will be ignored. Adjusting
                      this value helps reduce false positives.
  iou: 0.6           #An overlap threshold is used for non-maximum suppression (NMS).
                      Lower values ••reduce detections by eliminating overlapping bounding boxes.
  rect: True         #If enabled, the shorter side of the image is minimally padded
                      until it is divisible by the step size to improve inference speed.
```

```

half: True      #If enabled, half-precision inference is used, which can speed
up model inference on supported GPUs with minimal impact on accuracy.
device: "cuda:0"      #Reasoning device
batch : 50      #Batch size: Larger batch sizes can provide higher throughput,
thereby reducing the total time required for inference.
max_det : 4      #Maximum number of objects detected
augment : False   #Enabling Test-Time Enhancement (TTA) for prediction may
improve the robustness of detection, but it will reduce inference speed.
verbose : False   #Whether to output detailed inference information to the
terminal.

```

For normal debugging of road sign detection, only two parameters need to be changed: `conf: 0.92` (minimum confidence threshold, meaning that an object will only be selected if its similarity is higher than 0.92) and `max_det: 2` (maximum number of objects to detect, how many objects can be detected simultaneously). For detailed information on other parameters, please refer to the comments.

- rdkx5

```

#Road sign detection inference parameters
sign_detection:
  model_path:
    '/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/sign_model_
_bayese_640x640_nv12.bin'
  conf: 0.75      #Sets the minimum confidence threshold for detection.
Detected objects with a confidence level below this threshold will be ignored.
Adjusting this value helps reduce false positives.
  iou: 0.75      #An overlap threshold is used for non-maximum
suppression (NMS). Lower values reduce detections by eliminating overlapping
bounding boxes.
  classes_num: -1      #Number of categories, -1 indicates automatic detection
  reg: 16          #DFL reg layer
  strides: [8, 16, 32] #Model step size

line_detection:
#Lane detection inference parameters
  model_path:
    '/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/lane_v6_ba
yese_640x640_nv12.bin'
  conf: 0.45      #Sets the minimum confidence threshold for detection.
Detected objects with a confidence level below this threshold will be ignored.
Adjusting this value helps reduce false positives.
  iou: 0.65      #An overlap threshold is used for non-maximum
suppression (NMS). Lower values reduce detections by eliminating overlapping
bounding boxes.
  classes_num: 1      #Number of categories
  reg: 16          #DFL reg layer
  strides: [8, 16, 32] #Model step size

```

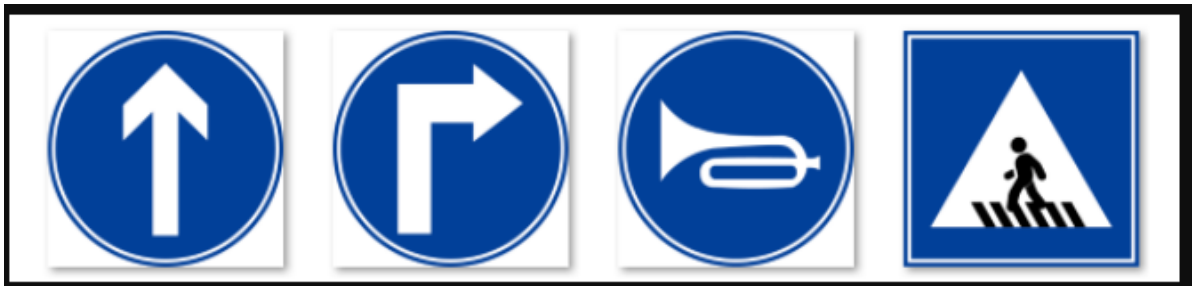
You can also set the confidence threshold by modifying the configuration file. **RDk doesn't have a `max_det` parameter**, so it doesn't need adjustment.

After modifying the parameters, recompile. The next time you run the application, it will use the modified parameters.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/  
colcon build --packages-select auto_drive
```

- Be prepared with road signs:

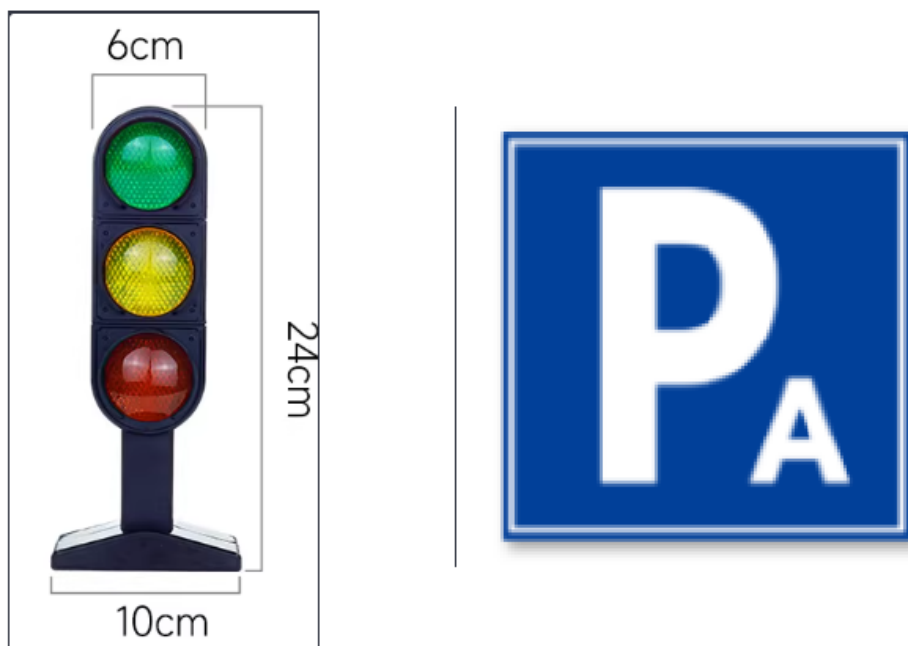
Go straight, turn right, honk your horn, pedestrian crossing



Speed limit, stop, school, parking lot B



Traffic light, parking lot A



3. Running the Case Program

Autonomous driving source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

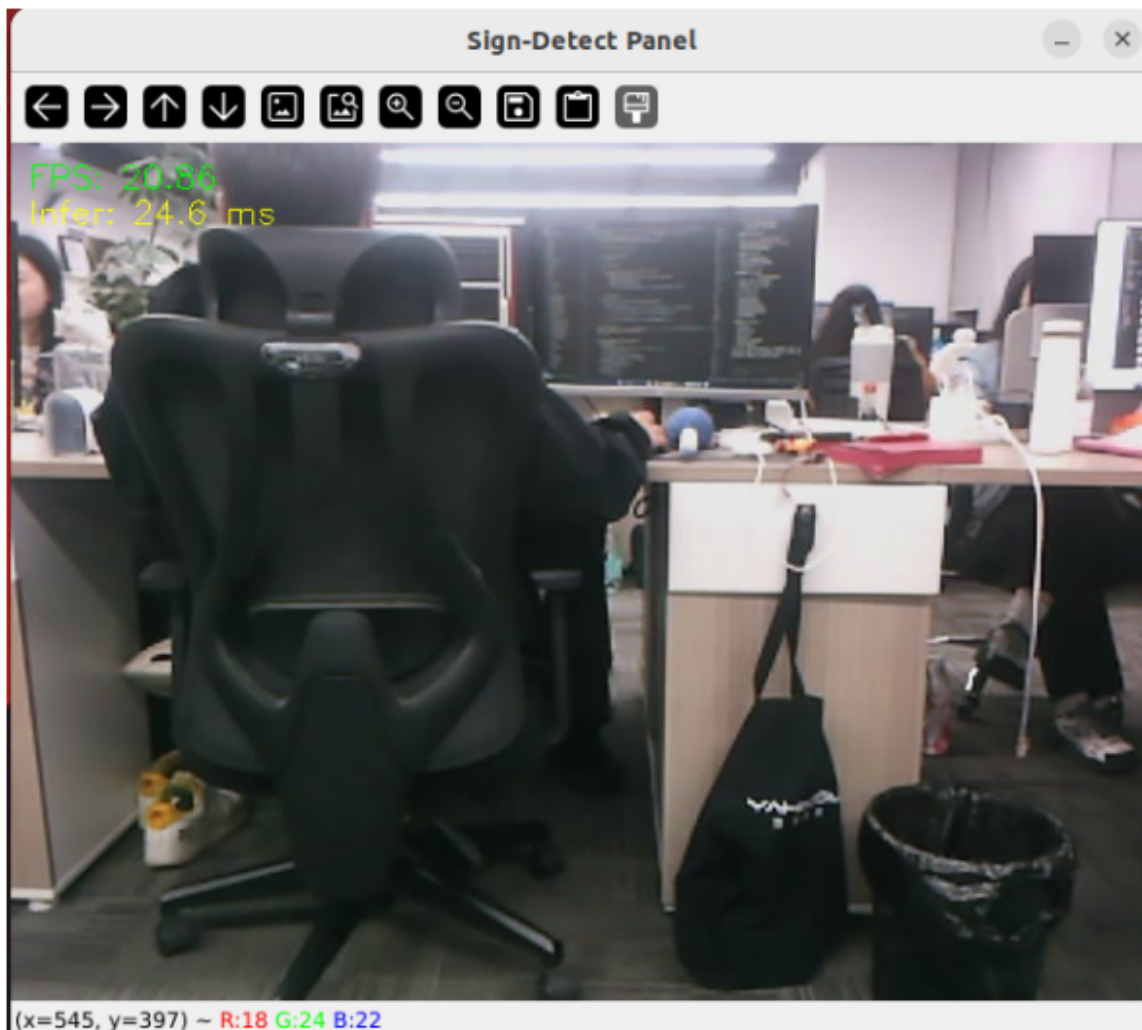
- Start the camera node

```
ros2 launch ascamera hp60c.launch.py
```

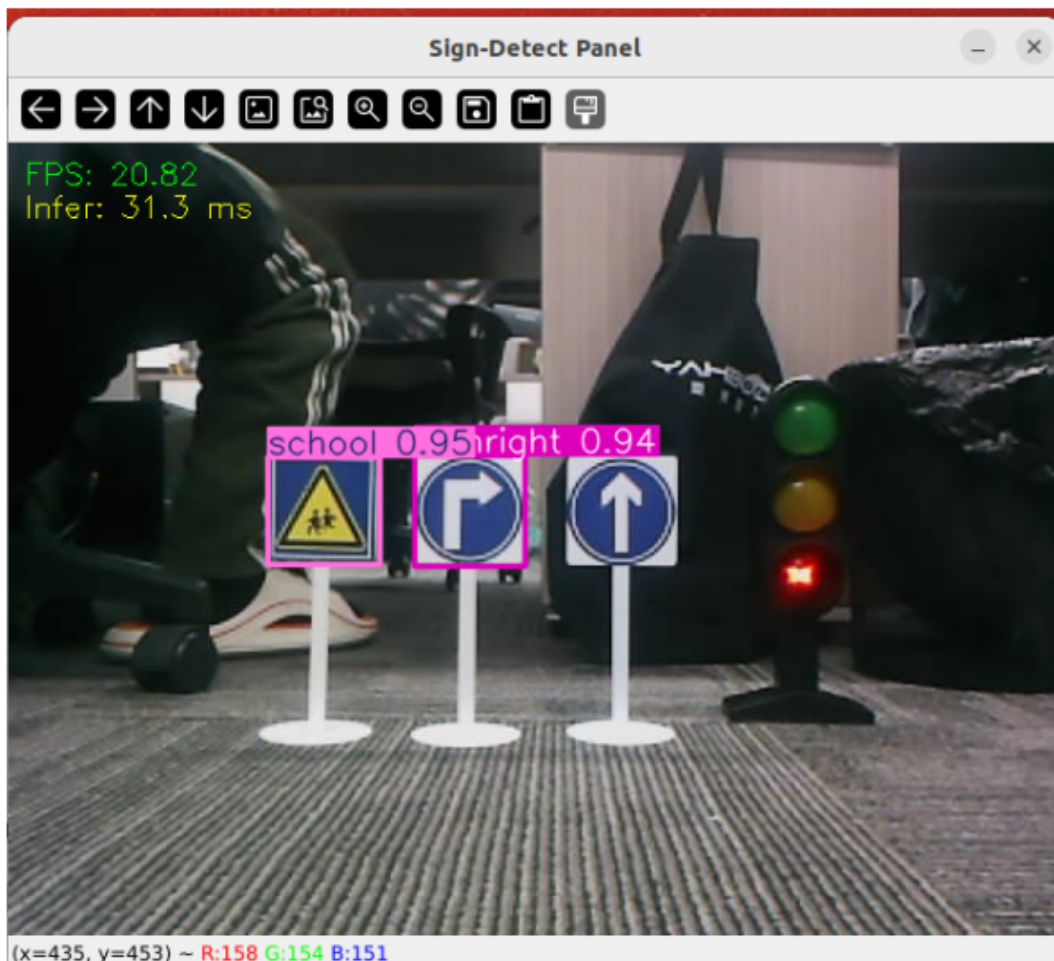
- The starting lane line is centered.

```
ros2 run auto_drive yolo_detect
```

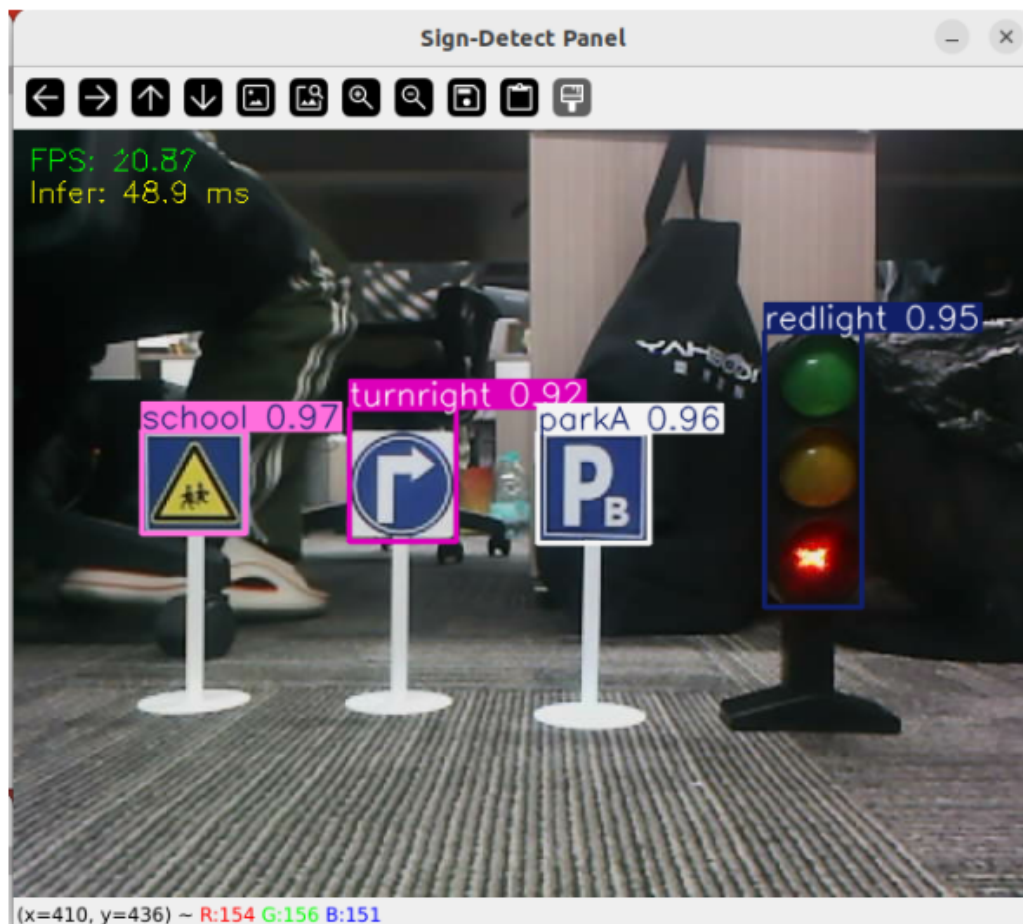
!



Placing a road sign in the center of the image will correctly identify its name. `max_det : 2` only recognizes two road signs at a time, which may lead to misidentification during autonomous driving. It is recommended to change `max_det : 2`.



max_det : 4 recognizes four road signs simultaneously.



保The road signs on the certified road can be recognized. The recognition rate will be even better if we get closer. We can proceed to the next step of debugging.

4. Source Code Analysis

4.1 yolo_detect.py

Program source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_detect.py
```

Control Flow

1. Image Callback:

- Convert ROS image messages to OpenCV format
- Add the image to the processing queue

2. Handle_sign Processing:

- Retrieve image frames from the queue
- Perform object detection using the YOLO model
- Process the detection results:
 - Filter out ignored signs (e.g., "turnright_ground", "sidewalk_ground")
 - Publish the detected sign information
 - Display FPS and inference time in debug mode
 - Display detection results in real time

The main thread is responsible for receiving images, and worker threads are responsible for processing images and detection tasks. Inter-thread communication is achieved through queues.

```
def handle_sign(self):
    '''处理图像中的道路标识检测结果 Processing road sign detection results in images'''
    while True:
        try:
            frame = self.sign_detect.image_queue.get(timeout=0.01)
        except queue.Empty:
            continue
        infer_start = time.time()
        results=self.sign_detect.get_infer_result(frame,True)
        for res in results:
            if res.bboxes is not None:
                annotated_frame = res.plot()
                boxes = res.bboxes
                for i in range(len(boxes)):
                    box_coords = boxes.xyxy[i].tolist()# Get the bounding
box coordinates (xyxy format: xmin, ymin, xmax, ymax)
                    class_name = res.names[int(boxes.cls[i].item())]# Get
object name
                    confidence = boxes.conf[i].item()# Obtain confidence

                    # Check if it is a flag that needs to be ignored.
                    if class_name in self.ignored_signs:
                        # rospy.loginfo(f"Ignoring sign: {class_name}")
                        continue # Skip the ignore flag

                # Release of test results
                if self.sign_pub_counter % 5 == 0:
```

```

        self.sign_publisher.publish(DetectionObject(
            class_name=class_name,
            confidence=confidence,
            xmin=float(box_coords[0]),
            ymin=float(box_coords[1]),
            xmax=float(box_coords[2]),
            ymax=float(box_coords[3])
        ))
    self.sign_pub_counter += 1

if self.DEBUG_MODE:
    infer_end = time.time()
    # Update FPS
    self.frame_count += 1
    curr_time = time.time()
    time_diff = curr_time - self.prev_time
    if time_diff >= 1.0:
        self.fps = self.frame_count / time_diff
        self.frame_count = 0
        self.prev_time = curr_time
    # Display FPS on the image
    cv2.putText(annotated_frame, f"FPS: {self.fps:.2f}", (10,
25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1,)
    # Display inference time
    infer_time = (infer_end - infer_start) * 1000
    cv2.putText(annotated_frame, f"Infer: {infer_time:.1f} ms", (10,
45), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

    cv2.imshow("Sign-Detect Panel", annotated_frame)
    cv2.waitKey(1)

```